

06_Spark_Rendimiento_y_Shuffles_executed

March 11, 2026

1 Práctica 3: Rendimiento, Shuffles y Joins (Medallón)

Dataset: US Accidents (Desde 2016, >7 Millones de filas) **Tecnología:** Apache Spark (Modo Local)

Los archivos CSV extraídos de base operativa en texto plano agotan procesadores y colapsan memorias. Apache inventó el formato **Parquet** que brilla entre tu capa Bronce y tu capa Plata.

Recuerda: Este dataset supera los 3GB. Tarda minutos en bajar de Kaggle (si falla, renueva tu token y evitar el Error 403).

```
[2]: import os
os.environ["SPARK_DRIVER_MEMORY"] = "4g"
os.environ["PYSPARK_DRIVER_PYTHON"] = "python3"
from pyspark.sql import SparkSession

# Le damos bastante memoria a nuestro driver para no sufrir con 7.7 millones de
  ↳filas brutas
spark = (SparkSession.builder
        .appName("Practica_Rendimiento_Spark_Medallion")
        .master("local[*]")
        .config("spark.sql.shuffle.partitions", "16")    # prueba 8, 16, 32
        .config("spark.driver.memory", "4g")
        .getOrCreate()
        )
# Ejemplo: muestra la versión de PySpark
try:
    print(f"Versión de PySpark {spark.version}.")
except NameError:
    print("SparkSession no inicializada; ejecuta la celda de creación de spark.
  ↳")
```

Versión de PySpark 3.5.0.

1.0.1 Bronce (Extract)

```
[2]: !pip install -q opendatasets
import opendatasets as od

dataset_url = "https://www.kaggle.com/datasets/sobhanmoosavi/us-accidents"
od.download(dataset_url)

!hdfs dfs -mkdir -p /data/bronze/accidents/
!hdfs dfs -put -f us-accidents/*.csv /data/bronze/accidents/
print("Descarga completada y carga a HDFS")
```

Descarga completada y carga a HDFS

1.0.2 Plata: El Test de Velocidad (CSV vs Parquet)

La estandarización principal duele leerla de Bronce porque es CSV. Observa los tiempos.

```
[3]: import time
ruta_csv = "hdfs://namenode:9000/data/bronze/accidents/*.csv"
ruta_parquet = "hdfs://namenode:9000/data/silver/accidents/"

# --- 1. Mide tiempo de inferir el CSV por defecto (Masoquismo) ---
t0 = time.time()
df_csv = spark.read.option("header", "true").option("inferSchema", "true").
    ↪csv(ruta_csv)
print(f"Líneas consumidas de Bronce: {df_csv.count()} (Tiempo bruto: {time.
    ↪time() - t0:.2f} segundos)")

# --- 2. Convertimos el Dataset Crudo limpio a Plata (En bloques binarios,
    ↪columnares) ---
# Usamos repartition(8) para distribuir la carga de memoria entre particiones
df_csv.repartition(8).write.mode("overwrite").parquet(ruta_parquet)

# --- 3. Mide tiempo de lectura de un Parquet nativo (La experiencia Plata) ---
t0 = time.time()
df_plata = spark.read.parquet(ruta_parquet)
print(f"Líneas consumidas de Plata: {df_plata.count()} (Tiempo veloz: {time.
    ↪time() - t0:.2f} segundos)")
```

Líneas consumidas de Bronce: 7728394 (Tiempo bruto: 49.48 segundos)

Líneas consumidas de Plata: 7728394 (Tiempo veloz: 0.85 segundos)

1.0.3 Oro: Shuffles Limitados y Particiones Controladas

Spark, por defecto divide los trabajos pesados de red de Oro (Shuffles, por ej un Group By para negocio) en **200 cajones (Particiones)**. A veces congestionará la red si lo hacemos mal.

```
[4]: # 1. Ajustamos la red
spark.conf.set("spark.sql.shuffle.partitions", "25")

# 2. Regla de Negocio (Oro): Agrupamos cruces de Estado y Gravedad (Lento pero
↳ necesario)
df_oro = df_plata.groupBy("State", "Severity").count()

# 3. Optimizador Final: Hacemos un `coalesce` (fusionar sin mover datos a
↳ través de la red)
# para que no nos queden 25 fragmentitos diminutos de parquet en HDFS Oro y
↳ solo tengamos 1 bloque.
df_oro_final = df_oro.coalesce(1)

# 4. Escribimos la maravilla a la tabla de reportes Oro
ruta_oro = "hdfs://namenode:9000/data/gold/accidents_kpi/"
df_oro_final.write.mode("overwrite").parquet(ruta_oro)
df_oro_final.show(10)
```

```
+-----+-----+-----+
|State|Severity| count|
+-----+-----+-----+
|  MI|      2|113020|
|  RI|      3| 7728|
|  MA|      3|18623|
|  VA|      3|51324|
|  NE|      2|24047|
|  MD|      1| 517|
|  NE|      1| 93|
|  SC|      1| 6175|
|  KY|      2|18255|
|  WI|      4| 3462|
+-----+-----+-----+
only showing top 10 rows
```

```
[5]: spark.stop()
```